

ARTIFICIAL INTELLIGENCE AND DEEP LEARNING

LAB ASSIGNMENT - 4

NAME – HARI RISHIKA KOMATI

ENROLLMENT NO – 2403A51083

BRANCH – CSE

YEAR/SEM – 3/1

DATE – 13-08-26

Lab/Exp 4: Implementing an ANN for Multi-class Classification -e.g., Softmax Regression

Kaggle Dataset Link: <https://www.kaggle.com/datasets/yasserh/wine-quality-dataset>

Tasks:

1. Load and preprocess the Wine Quality dataset/example.
 2. Build the required PyTorch model/program.
 3. Train/execute the model.
 4. Evaluate using suitable metrics.
 5. Visualize results using plots where applicable.
 6. Write a brief conclusion comparing observations.
-
-
-

CODE –

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

import torch
import torch.nn as nn
import torch.optim as optim

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report,
    confusion_matrix
)

from google.colab import files

# Check PyTorch version
print("PyTorch Version:", torch.__version__)

print("\nUpload the Wine Quality CSV file:")
uploaded = files.upload()

file_name = list(uploaded.keys())[0]

# Load dataset
df = pd.read_csv('/content/archive (4).zip')

print("\nDataset loaded successfully!")

print("\nFirst 5 rows:")
print(df.head())

print("\nDataset Shape:", df.shape)

# Remove unnecessary column if present
# WineQT.csv usually contains an 'Id' column
if "Id" in df.columns:
    df = df.drop("Id", axis=1)

# Remove missing values
df = df.dropna()

print("\nMissing Values:")
print(df.isnull().sum())

# Separate features and target
X = df.drop("quality", axis=1)
y = df["quality"]
```

```
print("\nFeature Columns:")
print(X.columns.tolist())

print("\nOriginal Target Classes:")
print(sorted(y.unique()))

print("\nClass Distribution:")
print(y.value_counts().sort_index())

# Convert quality values to class indices
# Example:
# 3 -> 0
# 4 -> 1
# 5 -> 2
# etc.

classes = sorted(y.unique())

class_mapping = {
    label: index for index, label in enumerate(classes)
}

y = y.map(class_mapping)

print("\nClass Mapping:")
print(class_mapping)

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("\nTraining Samples:", X_train.shape[0])
print("Testing Samples:", X_test.shape[0])

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

X_train_tensor = torch.tensor(
    X_train,
    dtype=torch.float32
)

X_test_tensor = torch.tensor(
    X_test,
    dtype=torch.float32
)
```

```

y_train_tensor = torch.tensor(
    y_train.values,
    dtype=torch.long
)

y_test_tensor = torch.tensor(
    y_test.values,
    dtype=torch.long
)

print("\nTensor Shapes:")
print("X_train:", X_train_tensor.shape)
print("X_test :", X_test_tensor.shape)
print("y_train:", y_train_tensor.shape)
print("y_test :", y_test_tensor.shape)

input_size = X_train.shape[1]
num_classes = len(classes)

class WineQualityANN(nn.Module):

    def __init__(self):
        super(WineQualityANN, self).__init__()

        self.network = nn.Sequential(

            # Input Layer -> Hidden Layer
            nn.Linear(input_size, 64),
            nn.ReLU(),

            # Hidden Layer
            nn.Linear(64, 32),
            nn.ReLU(),

            # Output Layer
            nn.Linear(32, num_classes)
        )

    def forward(self, x):
        return self.network(x)

# Create model
model = WineQualityANN()

print("\n=====")
print("ANN MODEL")
print("=====")
print(model)

criterion = nn.CrossEntropyLoss()

optimizer = optim.Adam(
    model.parameters(),

```



```
train_losses = []
train_accuracies = []

print("\n=====")
print("TRAINING STARTED")
print("=====")

for epoch in range(epochs):

    # Training mode
    model.train()

    # Forward propagation
    outputs = model(X_train_tensor)

    # Calculate loss
    loss = criterion([
        outputs,
        y_train_tensor
    ])

    # Clear old gradients
    optimizer.zero_grad()

    # Backpropagation
    loss.backward()

    # Update weights
    optimizer.step()

    # Calculate training accuracy
    → predicted = torch.max(
        outputs,
        1
    )

    accuracy = (
        predicted == y_train_tensor
    ).float().mean()

    # Store values
    train_losses.append(loss.item())
    train_accuracies.append(
        accuracy.item()
    )

    # Display progress
    if (epoch + 1) % 10 == 0:

        print(
            f"Epoch [{epoch+1:3d}/{epochs}] | "
            f"Loss: {loss.item():.4f} | "
            f"Accuracy: {accuracy.item()*100:.2f}%"
        )
```

```

print("\nTraining completed!")

model.eval()

with torch.no_grad():

    test_outputs = model(
        X_test_tensor
    )

    _, predictions = torch.max(
        test_outputs,
        1
    )

# Convert tensors to NumPy
y_true = y_test_tensor.numpy()
y_pred = predictions.numpy()

accuracy = accuracy_score(
    y_true,
    y_pred
)

precision = precision_score(
    y_true,
    y_pred,
    average="weighted",
    zero_division=0
)

recall = recall_score(
    y_true,
    y_pred,
    average="weighted",
    zero_division=0
)

f1 = f1_score(
    y_true,
    y_pred,
    average="weighted",
    zero_division=0
)

print("\n=====")
print("MODEL EVALUATION RESULTS")
print("=====")

print(
    f"Accuracy : {accuracy * 100:.2f}%"
)

```

```

print(
    f"Recall    : {recall * 100:.2f}%"
)

print(
    f"F1 Score  : {f1 * 100:.2f}%"
)

# -----
# 13. CLASSIFICATION REPORT
# -----

print("\n=====")
print("CLASSIFICATION REPORT")
print("=====")

print(
    classification_report(
        y_true,
        y_pred,
        target_names=[
            str(c) for c in classes
        ],
        zero_division=0
    )
)

# -----
# 14. CONFUSION MATRIX
# -----

cm = confusion_matrix(
    y_true,
    y_pred
)

plt.figure(figsize=(8, 6))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=classes,
    yticklabels=classes
)

plt.xlabel("Predicted Quality")
plt.ylabel("Actual Quality")
plt.title(
    "Confusion Matrix - Wine Quality ANN"
)

plt.show()

```

```

plt.figure(figsize=(8, 5))

plt.plot(
    range(1, epochs + 1),
    train_losses,
    linewidth=2
)

plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Training Loss vs Epochs")

plt.grid(True)

plt.show()

plt.figure(figsize=(8, 5))

plt.plot(
    range(1, epochs + 1),
    [x * 100 for x in train_accuracies],
    linewidth=2
)

plt.xlabel("Epoch")
plt.ylabel("Accuracy (%)")
plt.title("Training Accuracy vs Epochs")

plt.grid(True)

plt.show()

results = pd.DataFrame({
    "Actual Quality":
        [classes[i] for i in y_true],
    "Predicted Quality":
        [classes[i] for i in y_pred]
})

print("\n=====")
print("ACTUAL VS PREDICTED VALUES")
print("=====")

print(results.head(20))

sample = X_test_tensor[0].unsqueeze(0)

model.eval()

with torch.no_grad():
    output = model(sample)

```



```

▶ predicted_class = torch.argmax(
    output,
    dim=1
).item()

print("\n=====")
print("SAMPLE PREDICTION")
print("=====")

print(
    "Actual Wine Quality   :",
    classes[y_true[0]]
)

print(
    "Predicted Wine Quality :",
    classes[predicted_class]
)

# -----
# 19. FINAL CONCLUSION
# -----

print("\n=====")
print("CONCLUSION")
print("=====")

print("""
The Wine Quality dataset was successfully classified
using an Artificial Neural Network implemented in PyTorch.

The dataset was preprocessed by removing missing values,
encoding the target classes, and standardizing the input
features.

The ANN was trained using the Adam optimizer and
Cross-Entropy Loss. The model was evaluated using
Accuracy, Precision, Recall, F1-Score and a Confusion Matrix.

The training graphs show the change in loss and accuracy
during training. The results demonstrate that an ANN can
perform multi-class classification of wine quality, although
some quality classes may be difficult to distinguish because
their chemical properties are similar.
""")

print("=====")
print("LAB/EXPERIMENT COMPLETED")
print("=====")

```

OUTPUT:

Dataset loaded successfully!

First 5 rows:

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	\
0	7.4	0.70	0.00	1.9	0.076	
1	7.8	0.88	0.00	2.6	0.098	
2	7.8	0.76	0.04	2.3	0.092	
3	11.2	0.28	0.56	1.9	0.075	
4	7.4	0.70	0.00	1.9	0.076	

	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	\
0	11.0	34.0	0.9978	3.51	0.56	
1	25.0	67.0	0.9968	3.20	0.68	
2	15.0	54.0	0.9970	3.26	0.65	
3	17.0	60.0	0.9980	3.16	0.58	
4	11.0	34.0	0.9978	3.51	0.56	

	alcohol	quality	Id
0	9.4	5	0
1	9.8	5	1
2	9.8	5	2
3	9.8	6	3
4	9.4	5	4

Dataset Shape: (1143, 13)

Missing Values:

fixed acidity	0
volatile acidity	0
citric acid	0
residual sugar	0
chlorides	0
free sulfur dioxide	0
total sulfur dioxide	0
density	0
pH	0
sulphates	0
alcohol	0
quality	0
dtype:	int64

Feature Columns:

['fixed acidity', 'volatile acidity', 'citric acid', 'residual sugar', 'chlorides', 'free sulfur dioxide', 'total sulfur dioxide',

```

*** Original Target Classes:
[ np.int64(3), np.int64(4), np.int64(5), np.int64(6), np.int64(7), np.int64(8) ]

Class Distribution:
quality
3      6
4     33
5    483
6    462
7    143
8     16
Name: count, dtype: int64

Class Mapping:
{ np.int64(3): 0, np.int64(4): 1, np.int64(5): 2, np.int64(6): 3, np.int64(7): 4, np.int64(8): 5 }

Training Samples: 914
Testing Samples: 229

Tensor Shapes:
X_train: torch.Size([914, 11])
X_test : torch.Size([229, 11])
y_train: torch.Size([914])
y_test : torch.Size([229])

=====
ANN MODEL
=====
WineQualityANN(
  (network): Sequential(
    (0): Linear(in_features=11, out_features=64, bias=True)
    (1): ReLU()
    (2): Linear(in_features=64, out_features=32, bias=True)
    (3): ReLU()
    (4): Linear(in_features=32, out_features=6, bias=True)
  )
)

=====
TRAINING STARTED
=====
Epoch [ 10/100] | Loss: 1.7403 | Accuracy: 33.70%
Epoch [ 20/100] | Loss: 1.6040 | Accuracy: 44.75%
Epoch [ 30/100] | Loss: 1.4544 | Accuracy: 46.50%
Epoch [ 40/100] | Loss: 1.3090 | Accuracy: 51.64%
Epoch [ 50/100] | Loss: 1.1895 | Accuracy: 57.77%
Epoch [ 60/100] | Loss: 1.1078 | Accuracy: 57.44%
Epoch [ 70/100] | Loss: 1.0520 | Accuracy: 58.75%
Epoch [ 80/100] | Loss: 1.0106 | Accuracy: 60.18%
Epoch [ 90/100] | Loss: 0.9771 | Accuracy: 60.83%
Epoch [100/100] | Loss: 0.9516 | Accuracy: 61.60%

Training completed!

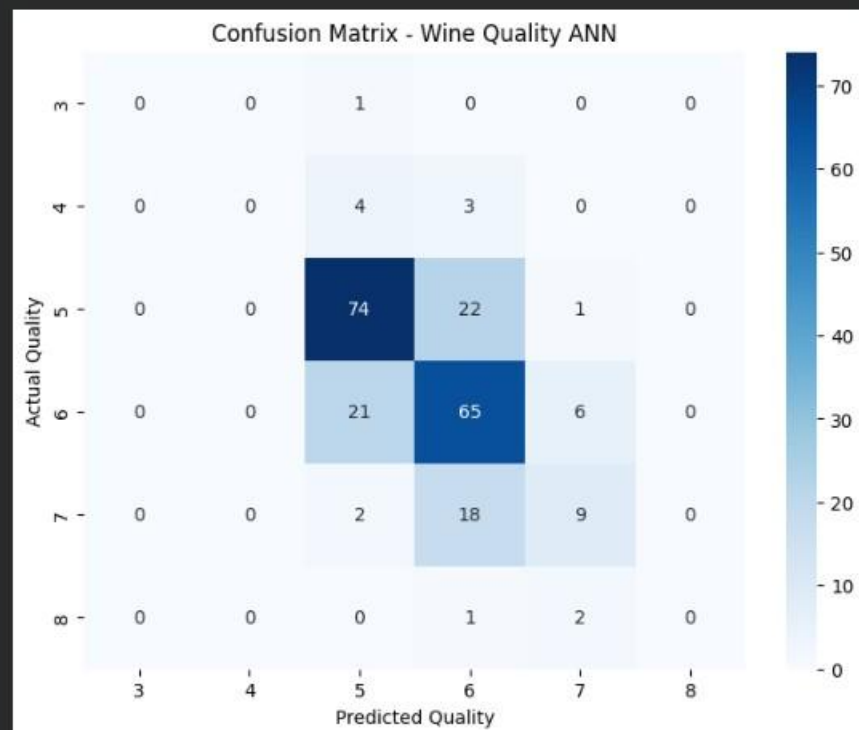
```

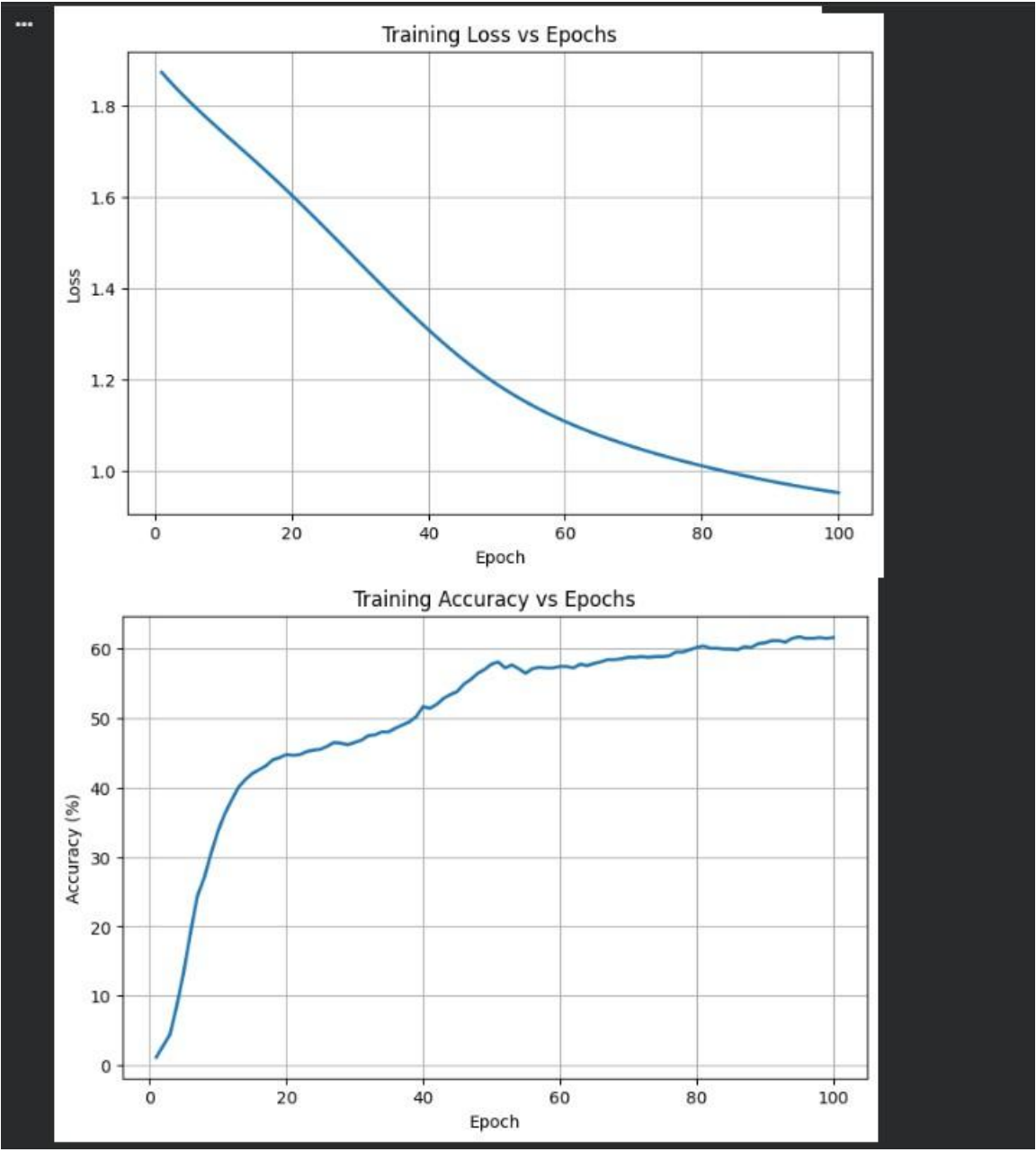
===== MODEL EVALUATION RESULTS =====

Accuracy : 64.63%
Precision : 61.02%
Recall : 64.63%
F1 Score : 62.34%

===== CLASSIFICATION REPORT =====

	precision	recall	f1-score	support
3	0.00	0.00	0.00	1
4	0.00	0.00	0.00	7
5	0.73	0.76	0.74	97
6	0.60	0.71	0.65	92
7	0.50	0.31	0.38	29
8	0.00	0.00	0.00	3
accuracy			0.65	229
macro avg	0.30	0.30	0.30	229
weighted avg	0.61	0.65	0.62	229





```
=====
ACTUAL VS PREDICTED VALUES
=====
```

	Actual Quality	Predicted Quality
0	6	6
1	5	5
2	5	5
3	5	5
4	5	6
5	6	6
6	6	6
7	6	6
8	6	6
9	4	5
10	5	5
11	5	5
12	7	6
13	6	6
14	7	7
15	5	5
16	5	5
17	6	6
18	5	6
19	5	5

```
=====
SAMPLE PREDICTION
=====
```

```
Actual Wine Quality   : 6
Predicted Wine Quality : 6
```

```
=====
CONCLUSION
=====
```

The Wine Quality dataset was successfully classified using an Artificial Neural Network implemented in PyTorch.

The dataset was preprocessed by removing missing values, encoding the target classes, and standardizing the input features.

The ANN was trained using the Adam optimizer and Cross-Entropy Loss. The model was evaluated using Accuracy, Precision, Recall, F1-Score and a Confusion Matrix.

The training graphs show the change in loss and accuracy during training. The results demonstrate that an ANN can perform multi-class classification of wine quality, although some quality classes may be difficult to distinguish because their chemical properties are similar.

```
=====
LAB/EXPERIMENT COMPLETED
=====
```